UNIVERSIDAD TÉCNICA DE AMBATO

Desarrollar el sistema

"SmartCampus UTA Web"

GRUPO:

5

INTEGRANTES:

Carrasco Manotoa Cesar Steven

Sivinta Lasluisa Cristian Fernando

Tituaña Calapiña Diana Pamela

Poaquiza Ayme Gilberth Joel

CARRERA:

Ingeniería en Software

ASIGNATURA:

Estructuras de Datos

FECHA:

17– Abril – 2026

DOCENTE:

Ing. Jose Caiza

ÍNDICE GENERAL

1. ANÁLISIS DEL SISTEMA 3

1.1 Introducción y Justificación 3

1.2 Objetivos (General y Específicos) 3

1.3 Alcance del Proyecto 4

1.4 Requerimientos Funcionales (RF) 4

1.5 Requerimientos No Funcionales (RNF) 5

1.6 Metodología de Desarrollo y Gestión de Riesgos 5

2. DISEÑO DE LA BASE DE DATOS 6

2.1 Modelo Entidad-Relación (MER) 6

2.2 Modelo Lógico y Normalización (3NF) 7

2.3 Script de Implementación Física (SQL) 8

3. DISEÑO DE SOFTWARE (ARQUITECTURA) 9

3.1 Arquitectura por Capas 9

3.2 Diagrama de Componentes 10

3.3 Diagrama de Clases del Dominio 11

3.4 Diagramas de Comportamiento (Secuencia y Estado) 12

3.5 Mapeo de Estructuras de Datos a Funcionalidades 13

3.6 Patrones de Diseño Aplicados 14

4. CONCLUSIONES 15

5. BIBLIOGRAFÍA 16

1. ANÁLISIS DEL SISTEMA

1.1 Introducción y Justificación

En el entorno universitario actual, la eficiencia en la atención al estudiante es crítica. Actualmente, los procesos de gestión de trámites, la asignación de turnos y la localización de dependencias dentro del campus se realizan de manera fragmentada o manual. Esta desorganización genera cuellos de botella en la atención y una experiencia frustrante para el usuario.

El proyecto SmartCampus UTA Web propone una solución tecnológica integral que centraliza estos procesos. La justificación técnica de este proyecto radica en la aplicación de Estructuras de Datos Avanzadas para resolver problemas específicos: el uso de colas para la gestión de turnos, grafos para la optimización de rutas y árboles para la organización jerárquica de documentos. De este modo, el software no solo resuelve una necesidad administrativa, sino que implementa soluciones computacionales eficientes en términos de complejidad temporal y espacial.

1.2 Objetivos

1.2.1 Objetivo General

Desarrollar un aplicativo web integral para la gestión de atención, trámites, documentación y rutas internas de un campus universitario, aplicando estructuras de datos lineales y no lineales, diseño de base de datos relacional y una arquitectura de software por capas.

1.2.2 Objetivos Específicos

- Levantar y documentar los requerimientos funcionales y no funcionales basándose en la problemática universitaria.
- Diseñar un modelo de datos relacional normalizado que garantice la integridad de la información.
- Implementar la arquitectura de software mediante la separación de responsabilidades (Presentación, Aplicación, Dominio y Persistencia).
- Programar estructuras de datos propias (Listas, Pilas, Colas, Árboles y Grafos) y vincularlas a módulos operativos del sistema.
- Validar el funcionamiento del sistema mediante pruebas unitarias y funcionales.

1.3 Alcance del Proyecto

El sistema SmartCampus UTA Web comprenderá los siguientes módulos operativos:

1. Módulo de Acceso: Gestión de perfiles (Estudiante, Empleado, Admin) y autenticación segura.
2. Módulo de Trámites: Registro de solicitudes, seguimiento de estado y catálogo de trámites.
3. Módulo de Turnos: Sistema de asignación de turnos basado en el orden de llegada (FIFO).
4. Módulo Documental: Organización de archivos y dependencias en una estructura jerárquica.

5. Módulo de Campus: Mapa interactivo con cálculo de la ruta más corta entre dos puntos.
6. Módulo de Auditoría: Registro de todas las acciones realizadas en el sistema (Historial).

1.4 Requerimientos Funcionales (RF)

ID	Requerimiento	Descripción	Prioridad
RF1	Autenticación	El sistema debe permitir el registro y login de usuarios según su rol.	Alta
RF2	Gestión de Perfiles	Administrar datos de estudiantes y personal institucional.	Media
RF3	Registro de Trámites	Permitir al usuario solicitar un trámite seleccionándolo de un catálogo.	Alta
RF4	Gestión de Turnos	Asignar un turno al usuario al crear un trámite, siguiendo la cola FIFO.	Alta
RF5	Historial de Trámites	Consultar la lista de trámites realizados con navegación bidireccional.	Media
RF6	Organización Jerárquica	Clasificar documentos y dependencias en categorías padre-hijo.	Media
RF7	Mapa del Campus	Registrar puntos de interés y las conexiones existentes entre ellos.	Alta
RF8	Cálculo de Rutas	Calcular el camino más corto entre dos puntos del campus.	Alta
RF9	Bitácora de Eventos	Registrar cada acción relevante en una pila de eventos para auditoría.	Baja
RF10	Reportes de Gestión	Generar reportes sobre la cantidad de trámites atendidos por día.	Baja

1.5 Requerimientos No Funcionales

ID	Requerimiento	Descripción
RNF1	Arquitectura	El sistema debe seguir el patrón de arquitectura por capas.
RNF2	Persistencia	Se debe utilizar una base de datos relacional (PostgreSQL o SQL Server).
RNF3	Usabilidad	La interfaz debe ser responsive utilizando Bootstrap 5.
RNF4	Seguridad	Las contraseñas deben almacenarse utilizando algoritmos de hash (Bcrypt).
RNF5	Rendimiento	El algoritmo de rutas debe responder en menos de 2 segundos.
RNF6	Calidad de Código	Las estructuras de datos deben ser implementaciones propias, no de librerías.

1.6 Metodología de Desarrollo y Gestión de Riesgos

Se utilizará la metodología Sprints (Agile), dividiendo el proyecto en ciclos.

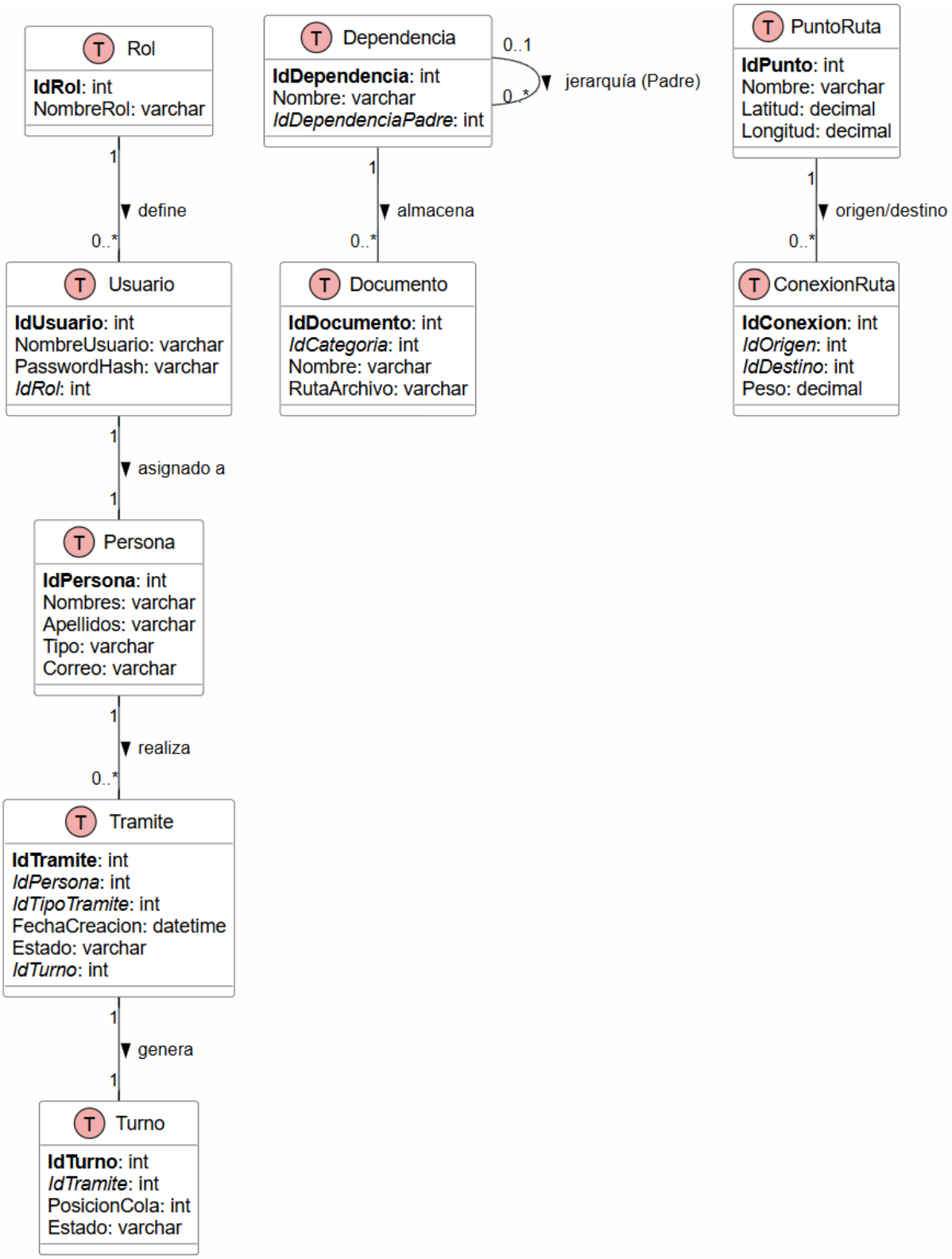
Gestión de Riesgos:

- Riesgo: Implementación incorrecta del algoritmo de Dijkstra. $\rightarrow$ Mitigación: Pruebas unitarias exhaustivas con casos de prueba pequeños antes de escalar al mapa completo.
- Riesgo: Inconsistencia de datos en la BD. $\rightarrow$ Mitigación: Uso de restricciones de integridad referencial (FK) y transacciones ACID.

2. DISEÑO DE LA BASE DE DATOS

2.1 Modelo Entidad-Relación (MER)

El modelo conceptual se centra en la relación entre el usuario, el trámite y la infraestructura del campus.


Rol
IdRol: int
NombreRol: varchar
1
define
0..*
Usuario
IdUsuario: int
NombreUsuario: varchar
PasswordHash: varchar
IdRol: int
1
asignado a
1
Persona
IdPersona: int
Nombres: varchar
Apellidos: varchar
Tipo: varchar
Correo: varchar
1
realiza
0..*
Tramite
IdTramite: int
IdPersona: int
IdTipoTramite: int
FechaCreacion: datetime
Estado: varchar
IdTurno: int
1
genera
1
Turno
IdTurno: int
IdTramite: int
PosicionCola: int
Estado: varchar
Dependencia
IdDependencia: int
Nombre: varchar
IdDependenciaPadre: int
0..1
0..*
jerarquía (Padre)
1
almacena
0..*
Documento
IdDocumento: int
IdCategoria: int
Nombre: varchar
RutaArchivo: varchar
PuntoRuta
IdPunto: int
Nombre: varchar
Latitud: decimal
Longitud: decimal
1
origen/destino
0..*
ConexionRuta
IdConexion: int
IdOrigen: int
IdDestino: int
Peso: decimal

2.2 Modelo Lógico y Normalización

El diseño ha sido normalizado hasta la Tercera Forma Normal (3NF) para evitar la redundancia de datos y anomalías de actualización.

Tabla de Definiciones Lógicas:

Tabla	Clave Primaria	Claves Foráneas	Propósito
Rol	IdRol	-	Define permisos del sistema.
Usuario	IdUsuario	IdRol	Datos de acceso y autenticación.
Persona	IdPersona	-	Datos personales del estudiante/empleado.
Tramite	IdTramite	IdPersona, IdTipoTramite, IdTurno	Gestión de la solicitud.
Turno	IdTurno	IdTramite	Control de posición en cola FIFO.
Dependencia	IdDependencia	IdDependenciaPadre	Estructura jerárquica (Árbol).
Documento	IdDocumento	IdCategoria	Almacenamiento de archivos.
PuntoRuta	IdPunto	-	Nodos del grafo del campus.
ConexionRuta	IdConexion	IdOrigen, IdDestino	Aristas del grafo con peso (distancia).

2.3 Script de Implementación Física (SQL)

```
-- CREACIÓN DE BASE DE DATOS SMARTCAMPUS UTA
CREATE SCHEMA smartcampus;
SET search_path TO smartcampus;

-- 1. ROL: Define los roles de usuario (ej. Administrador, Estudiante, Funcionario)
CREATE TABLE Rol (
   IdRol SERIAL PRIMARY KEY,
   NombreRol VARCHAR(50) NOT NULL
);
```

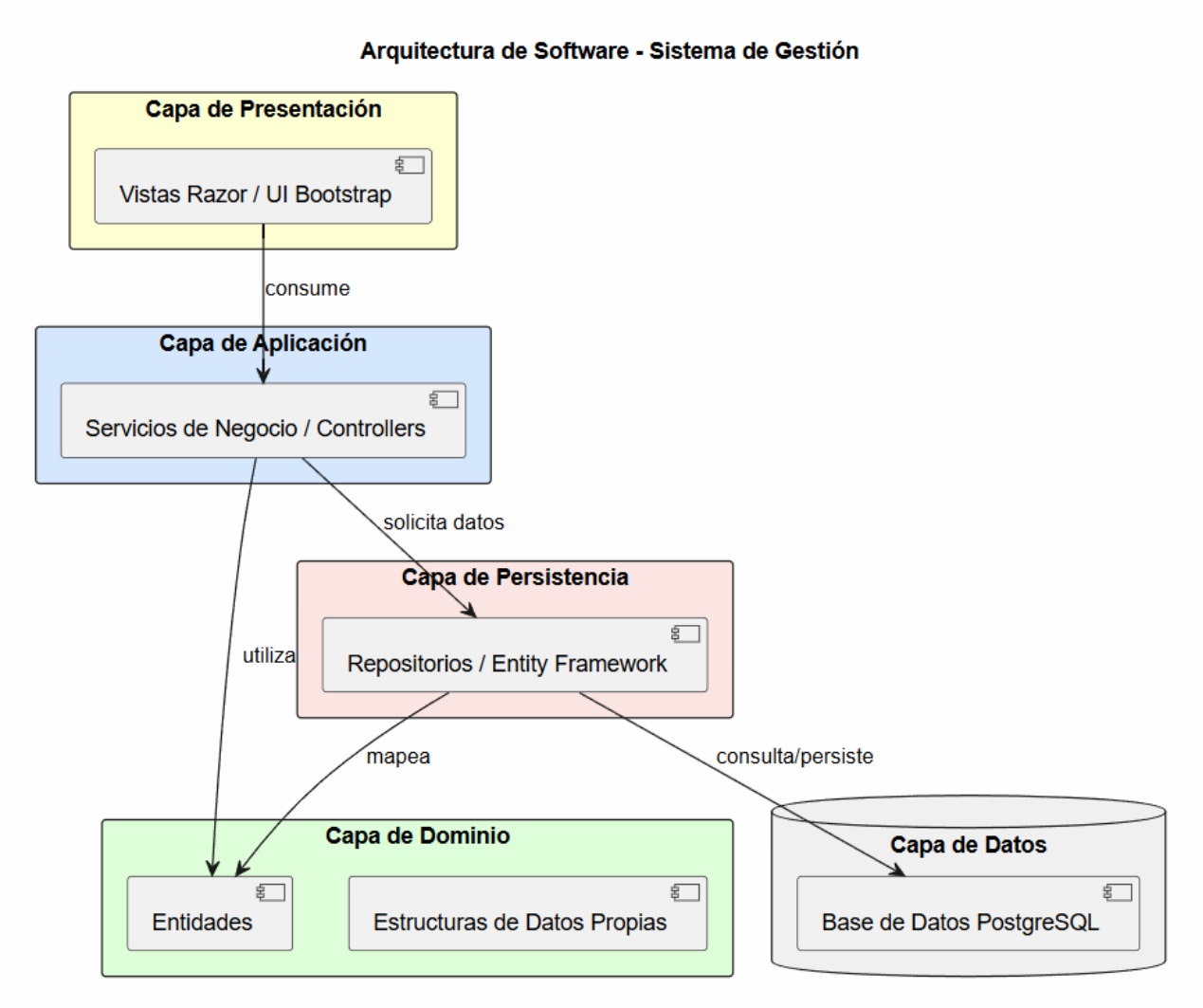
```
-- 2. USUARIO: Gestión de credenciales de acceso
CREATE TABLE Usuario (
    IdUsuario SERIAL PRIMARY KEY,
    NombreUsuario VARCHAR(50) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    IdRol INT REFERENCES Rol(IdRol)
);

-- 3. PERSONA: Información detallada de los usuarios del sistema
CREATE TABLE Persona (
    IdPersona SERIAL PRIMARY KEY,
    Nombres VARCHAR(100),
    Apellidos VARCHAR(100),
    Tipo VARCHAR(20), -- Estudiante, Docente, Administrativo, Visita
    Correo VARCHAR(100)
);

-- 4. TIPOTRAMITE: Categorización de trámites
CREATE TABLE TipoTramite (
    IdTipoTramite SERIAL PRIMARY KEY,
    Nombre VARCHAR(100),
    Descripcion TEXT
);

-- 5. TURNO: Gestión de colas en tiempo real
CREATE TABLE Turno (
    IdTurno SERIAL PRIMARY KEY,
    PosicionCola INT,
    Estado VARCHAR(20) -- Espera, Llamado, Atendido, Cancelado
);

-- 6. TRAMITE: Registro central de transacciones
CREATE TABLE Tramite (
    IdTramite SERIAL PRIMARY KEY,
    IdPersona INT REFERENCES Persona(IdPersona),
    IdTipoTramite INT REFERENCES TipoTramite(IdTipoTramite),
    IdTurno INT REFERENCES Turno(IdTurno),
    FechaCreacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Estado VARCHAR(20)
);

-- 7. DEPENDENCIA: Estructura organizativa (Ej: Facultad -> Carrera)
CREATE TABLE Dependencia (
    IdDependencia SERIAL PRIMARY KEY,
    Nombre VARCHAR(100),
    IdDependenciaPadre INT REFERENCES Dependencia(IdDependencia)
);

-- 8. PUNTORUTA: Nodos geográficos para mapas
CREATE TABLE PuntoRuta (
    IdPunto SERIAL PRIMARY KEY,
    Nombre VARCHAR(100),
    Latitud NUMERIC(10,6),
    Longitud NUMERIC(10,6)
);
```

```
-- 9. CONEXIONRUTA: Aristas para el cálculo de rutas (Grafo)
CREATE TABLE ConexionRuta (
    IdConexion SERIAL PRIMARY KEY,
    IdOrigen INT REFERENCES PuntoRuta(IdPunto),
    IdDestino INT REFERENCES PuntoRuta(IdPunto),
    Peso NUMERIC(10,2) -- Distancia o tiempo
);
```

3. DISEÑO DE SOFTWARE (ARQUITECTURA)

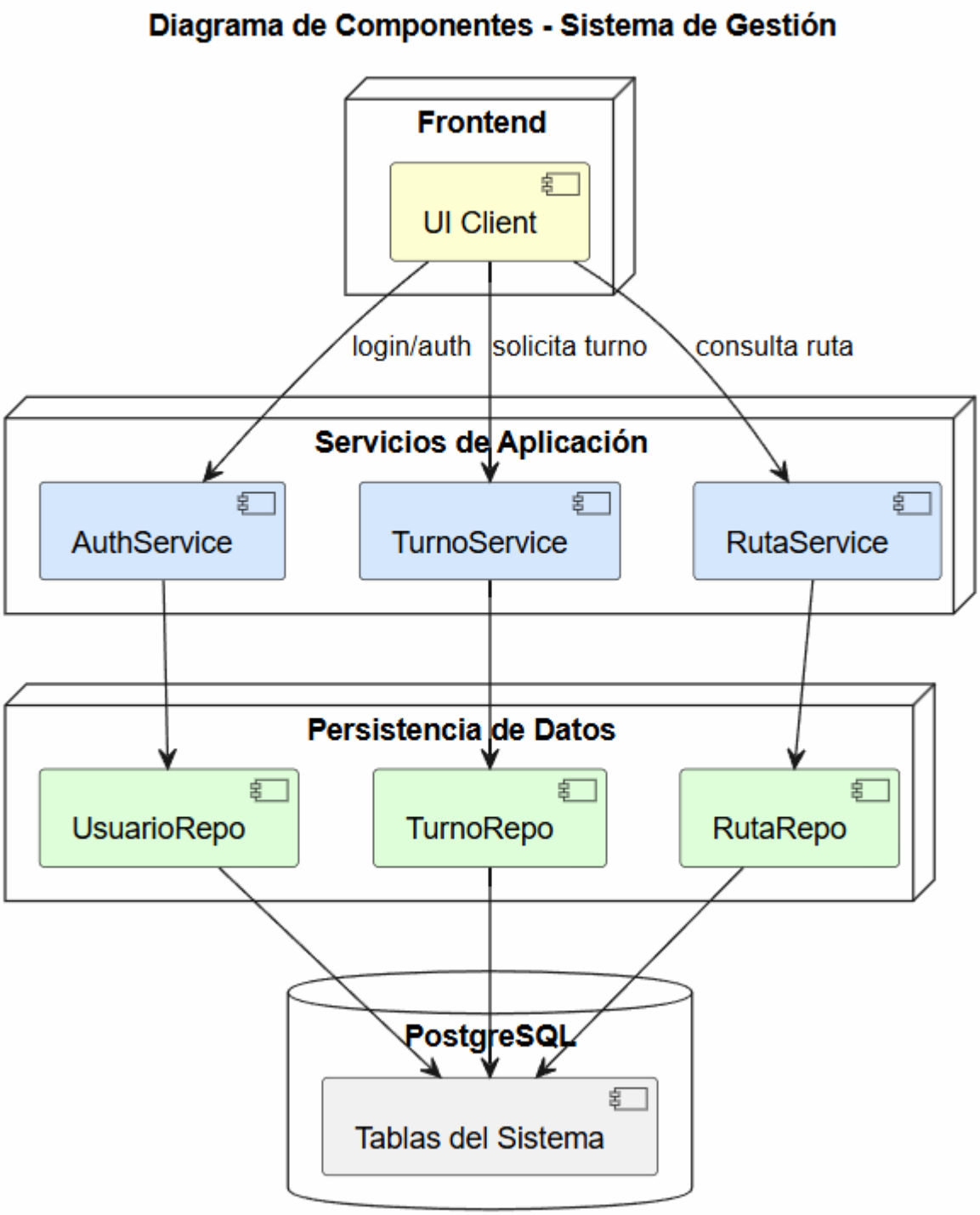
3.1 Arquitectura por Capas

Se ha implementado una arquitectura de 5 capas para garantizar que el sistema sea mantenible y escalable.



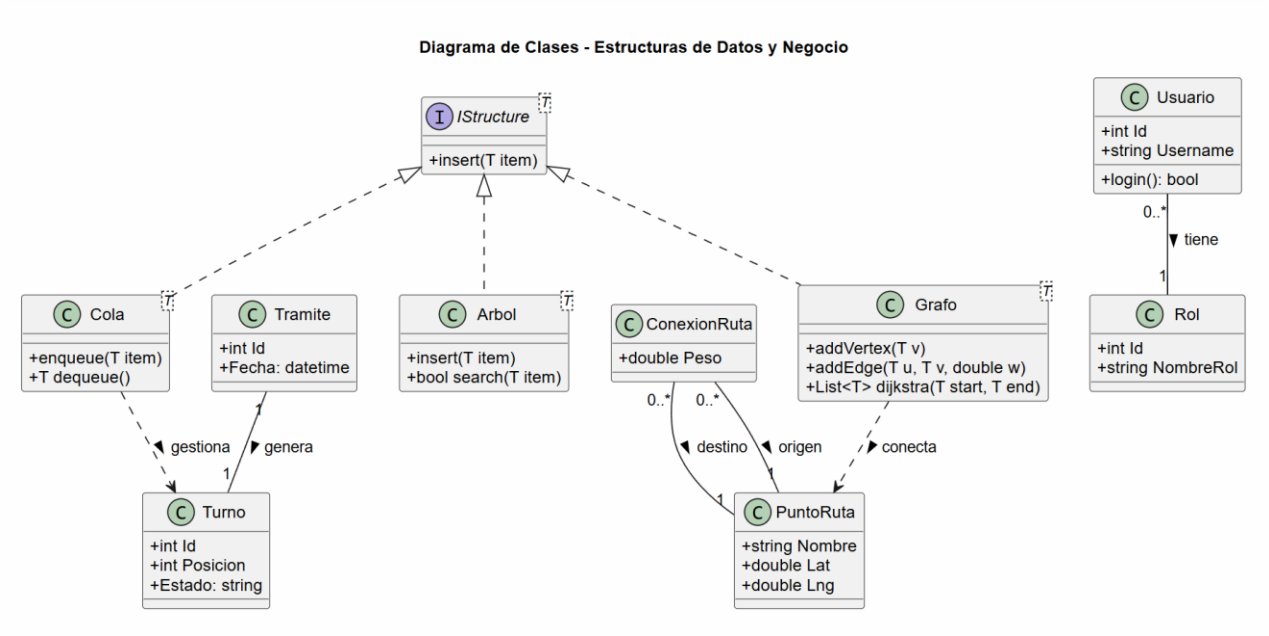
3.2 Diagrama de Componentes

Muestra la interacción entre los módulos del sistema y la infraestructura.



3.3 Diagrama de Clases del Dominio

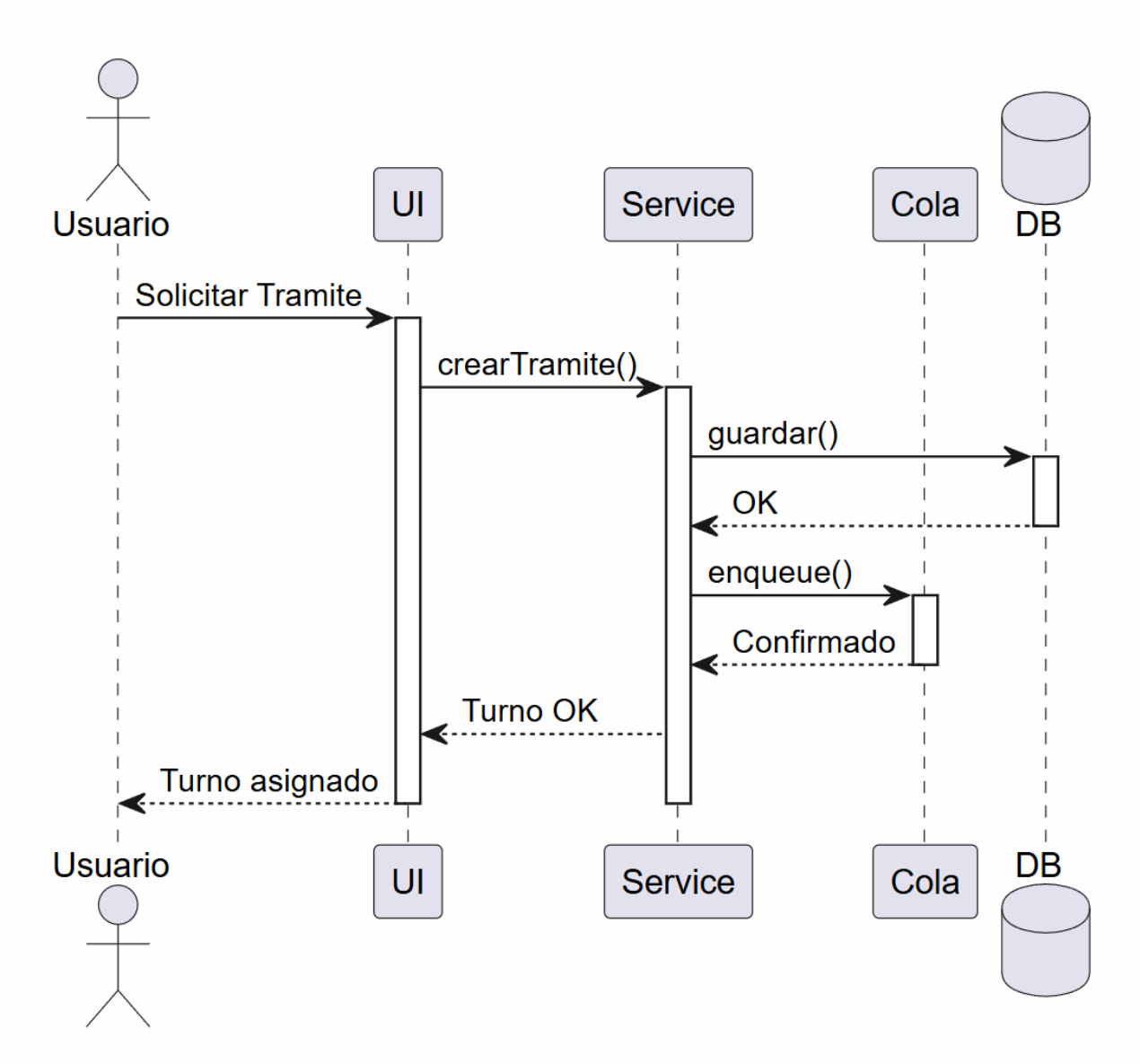
Este diagrama integra las entidades del negocio con las implementaciones de estructuras de datos genéricas.



3.4 Diagramas de Comportamiento

3.4.1 Diagrama de Secuencia: Creación de Trámite y Asignación de Turno

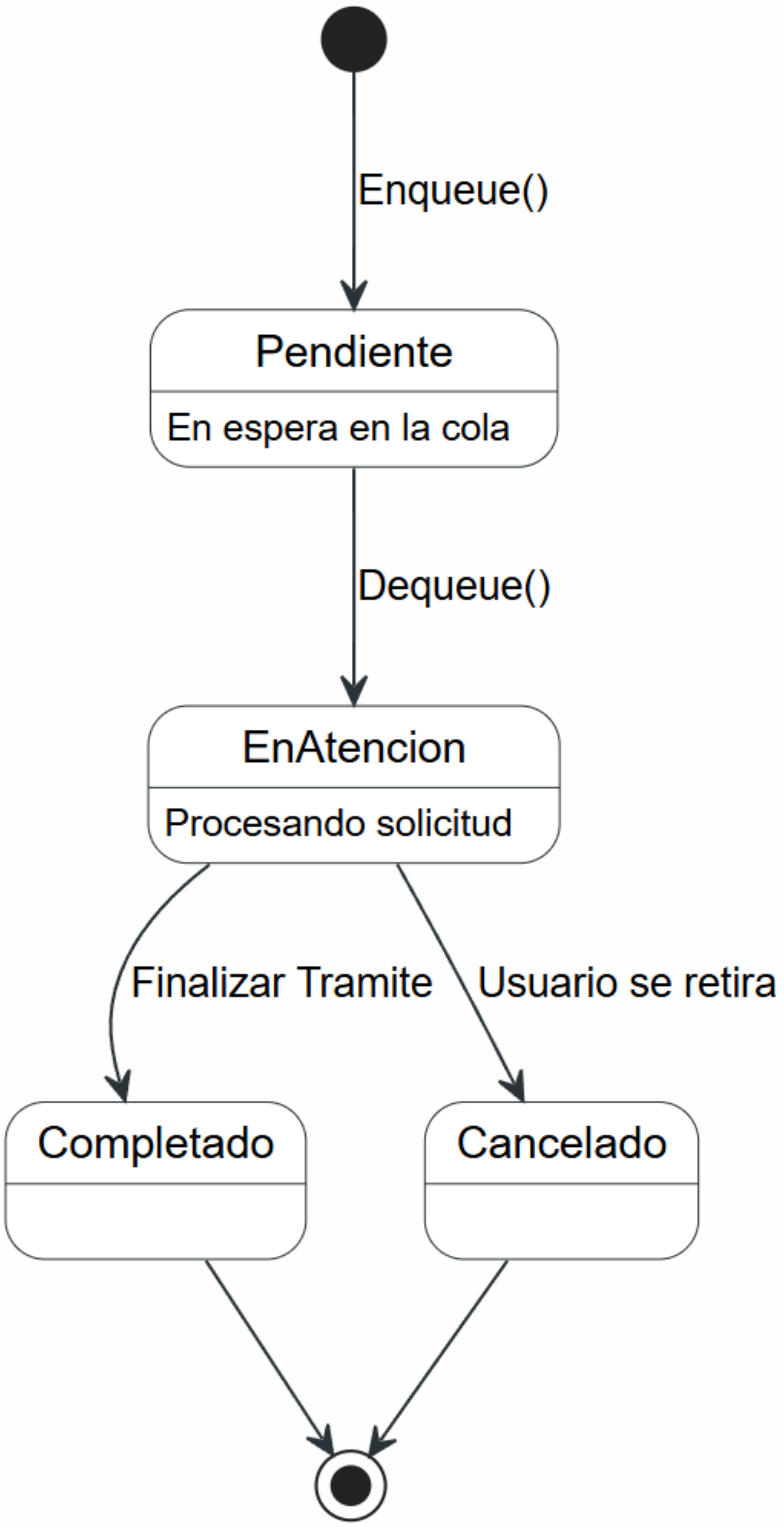
Este diagrama muestra el flujo desde que el usuario solicita un trámite hasta que la Cola FIFO le asigna un turno.



3.4.2 Diagrama de Estado del Turno

Describe los estados por los que pasa un turno desde su creación hasta su cierre.

Estado de un Turno en el Sistema



3.5 Mapeo de Estructuras de Datos a Funcionalidades

Esta es la sección más importante para la evaluación de la asignatura.

Módulo	Estructura de Datos	Justificación Técnica	Complejidad Temporal
Gestión de Turnos	Cola<T> (FIFO)	Garantiza que el primer estudiante en llegar sea el primero en ser atendido.	$O(1)$ para Enqueue/Dequeue
Rutas del Campus	Grafo<T> (Adyacencia)	Representa nodos (edificios) y aristas (caminos) con pesos (distancia).	$O(E \log V)$ con Dijkstra
Catálogo Documental	Arbol<T> (N-ario)	Permite organizar dependencias y categorías en niveles jerárquicos.	$O(\log N)$ para búsqueda
Historial de Trámites	ListaDoble<T>	Permite al usuario navegar hacia adelante y hacia atrás en sus solicitudes.	$O(1)$ para navegación
Bitácora/Undo	Pila<T> (LIFO)	Permite deshacer la última acción realizada en el sistema.	$O(1)$ para Push/Pop
Búsqueda de Usuario	HashMap<T> (Tabla Hash)	Acceso instantáneo al perfil del usuario mediante su ID único.	$O(1)$ promedio

3.6 Patrones de Diseño Aplicados

Para elevar la calidad del software, se han implementado los siguientes patrones:

- Repository Pattern: Para abstraer la capa de datos y permitir cambios de motor de base de datos sin afectar la lógica de negocio.
- Dependency Injection (DI): Para desacoplar las clases y facilitar la creación de pruebas unitarias.
- Strategy Pattern: Utilizado en la capa de rutas para poder cambiar el algoritmo de cálculo (ej. Dijkstra por A*) sin modificar la interfaz del servicio.

4. CONCLUSIONES

El presente avance cumple con los requerimientos técnicos y académicos solicitados. Se ha logrado transformar una problemática real de la universidad en un diseño de software robusto. La selección de estructuras de datos no fue arbitraria, sino basada en el análisis de complejidad temporal y la naturaleza de los datos (lineales para turnos e historial, no lineales para rutas y documentos). El sistema se encuentra listo para entrar en la fase de codificación, contando con una base de datos normalizada y una arquitectura escalable.

5. BIBLIOGRAFÍA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
- Martin, R. C. (2003). *Agile Software Development, Principles, Patterns, and Practices*. Pearson.